HO CHI MINH UNIVERSITY OF SCIENCE

FACULTY OF INFORMATION TECNOLOGY

DATA VISUALIZATION

LAB 1 REPORT

LECTURER:
Võ Nhật Tân

STUDENT (GROUP 2):
Nguyễn Đỗ Bảo - 23127026
Lê Nguyên Thảo - 23127118
Đinh Đại Vũ - 23127144
Vũ Hoàng Minh - 23127427
Nguyễn Thanh Owen - 23127447

Contents

1 Introduction to the Analytical Problems **5**
- 1.1 Domain Situation . . . 5
- 1.2 Analytical Objectives . . . 5
  - 1.2.1 Sales Performance by Category and Time . . . 5
  - 1.2.2 Pricing and Discount Strategy . . . 5
- 1.3 Product Influence Analysis . . . 6
  - 1.3.1 Customer Ratings and Review Signals . . . 7
  - 1.3.2 Segment Clustering and Revenue Contribution . . . 8

2 Data Collection, Construction, and Preprocessing **9**
- 2.1 Product Collection . . . 9
  - 2.1.1 Objectives and Approach . . . 9
  - 2.1.2 Detailed Execution Steps . . . 9
- 2.2 Review Collection . . . 10
  - 2.2.1 Objectives and Approach . . . 10
- 2.3 Detailed Execution Steps . . . 10
- 2.4 Data Preparation . . . 11
  - 2.4.1 Product Metadata Dataset . . . 11
  - 2.4.2 Data Cleaning and Standardization For Product Dataset . . . 12
  - 2.4.3 Customer Reviews Dataset . . . 13
  - 2.4.4 Data Cleaning and Standardization For Customer Review Dataset . . . 13

3 Data Analysis **14**

References **15**

List of Figures

List of Tables

1 List of group members 4
2 Task distribution and completion status 4

Group Members

No.	Full Name	Student ID	Email
1	Nguyễn Đỗ Bảo	23127026	ndbao23@clc.fitus.edu.vn
2	Lê Nguyên Thảo	23127118	lnthao23@clc.fitus.edu.vn
3	Đinh Đại Vũ	23127144	ddvu23@clc.fitus.edu.vn
4	Vũ Hoàng Minh	23127237	vhminh23@clc.fitus.edu.vn
5	Nguyễn Thanh Owen	23127451	ntowen23@clc.fitus.edu.vn

Table 1: List of group members

Task Distribution and Completion Status

No.	Task	Responsible	Student ID	Contribution
1		Nguyễn Đỗ Bảo	23127131	20%
2		Lê Nguyên Thảo	23127118	20%
3		Đinh Đại Vũ	23127144	20%
4		Vũ Hoàng Minh	23127427	20%
5		Nguyễn Thanh Owen	23127447	20%

Table 2: Task distribution and completion status

1 Introduction to the Analytical Problems

1.1 Domain Situation

1.2 Analytical Objectives

1.2.1 Sales Performance by Category and Time

- **Objective 1**: Measure how total revenue is distributed across Nike product categories from February 20, 2026 to March 20, 2026 in order to identify the highest-contributing categories and assess whether sales performance is concentrated in a small number of categories.
  - **Specific**: Category-level revenue contribution analysis aligned with the KPI cards and the `Total Revenue by Category name` bar chart.
  - **Measurable**: Uses `Total Daily Revenue`, `Average Revenue per Day`, `Revenue Difference`, and `Total Revenue by category_name`.
  - **Achievable**: All required revenue measures and category fields are available in the prepared sales dataset and current dashboard design.
  - **Relevant**: Directly supports the group's analytical problem by showing how product category influences sales performance.
  - **Time-bound**: Evaluated within the analytical window from 20-02-2026 to 20-03-2026.
- **Objective 2**: Compare the most volatile and most stable Nike product categories over February 20, 2026 to March 20, 2026 in order to determine which categories show consistent sales performance over time and which categories experience stronger revenue fluctuations.
  - **Specific**: Time-based category stability analysis using the `Top 3 Most Volatile Categories` and `Top 3 Most Stable Categories` visuals.
  - **Measurable**: Uses `Category Revenue CV` to compare the relative revenue fluctuation of categories across the analytical period.
  - **Achievable**: The required category-level revenue variation metric is available in the prepared model outputs and dashboard visuals.
  - **Relevant**: Directly supports the analytical problem by showing how category-level sales performance changes over time, not just in total size.
  - **Time-bound**: Evaluated within the same analytical window from 20-02-2026 to 20-03-2026.

1.2.2 Pricing and Discount Strategy

- **Objective 1**: Measure revenue-share and unit-share differences across predefined Price Band groups from February 20, 2026 to March 20, 2026 in order to identify the dominant price band in the current dashboard design.

- **Specific**: Price-band contribution analysis aligned with the `Price distribution` visual.
- **Measurable**: Uses share of `daily_revenue`, share of `daily_units_sold`, and `average selling_price` by Price Band.
- **Achievable**: `selling_price` and sales variables are available in the prepared snapshot dataset.
- **Relevant**: Directly supports interpretation of the KPI cards and pie-chart structure on the dashboard.
- **Time-bound**: Evaluated within the analytical window from 20-02-2026 to 20-03-2026.

- **Objective 2**: Build a chart-based comparison (line/column with slicers) of KPI changes under date and category filters from February 20, 2026 to March 20, 2026 in order to determine whether sales performance is broad-based or concentrated by segment.
  - **Specific**: Chart-based sensitivity analysis of `daily_revenue`, `daily_units_sold`, and `Average Selling Price` under slicer conditions.
  - **Measurable**: Uses KPI percentage change shown on the chart across selected date/category states and compares resulting segment composition.
  - **Achievable**: The required KPIs, segment dimensions, and filters are available in the current dashboard and source tables.
  - **Relevant**: Matches the interactive dashboard workflow and provides visual evidence for report/viva interpretation.
  - **Time-bound**: Limited to the same analytical period from 20-02-2026 to 20-03-2026.

1.3 Product Influence Analysis

- **Objective 1**: Identify the top products and categories driving daily unit sales from February 20, 2026 to March 20, 2026 using the current Top products visual and dashboard filters.
  - **Specific**: Product-level and category-level contribution analysis focused on unit performance.
  - **Measurable**: Uses `daily_units_sold` ranking, top-product share of total units sold, and category composition of top products.
  - **Achievable**: `Product name`, `category`, and `daily unit` fields are available in the current dashboard model.
  - **Relevant**: Directly supports the page purpose of understanding product influence on sales outcomes.
  - **Time-bound**: Evaluated within the analysis window from 20-02-2026 to 20-03-2026.

- **Objective 2**: Analyze time-based spikes and concentration risk among top products from February 20, 2026 to March 20, 2026 by combining trend visuals with date/category slicers.
  - **Specific**: Temporal sensitivity analysis of top-product unit performance under interactive filters.
  - **Measurable**: Uses peak `daily_units_sold` values by date, spike frequency of top products, and concentration ratio of top products versus total units sold.
  - **Achievable**: `Date`, `product`, `category`, and unit metrics are available in the current dashboard visuals.
  - **Relevant**: Explains whether page-level performance is broad-based or overly dependent on a small number of products.
  - **Time-bound**: Limited to the same analytical period from 20-02-2026 to 20-03-2026.

1.3.1 Customer Ratings and Review Signals

- **Objective 1**: Use the line chart of review count by year to examine how customer review activity changes across 2024, 2025, and 2026, in order to identify whether engagement with Nike products on Lazada is increasing, decreasing, or remaining stable over time.
  - **Specific**: Year-based review activity analysis using `reviews_count` across the three observed periods.
  - **Measurable**: Uses the trend and year-to-year differences in `reviews_count` from 2024 to 2026.
  - **Achievable**: The required review `date` and `review count` fields are available in the prepared review dataset and dashboard design.
  - **Relevant**: Supports the analytical problem by showing how customer review behavior evolves over time, which may relate to product visibility and sales performance.
  - **Time-bound**: Evaluated across the yearly periods 2024, 2025, and 2026.
- **Objective 2**: Use the bar chart of ratings by year to compare customer evaluation levels across 2024, 2025, and 2026, in order to determine whether Nike product ratings remain consistent or show meaningful changes over time.
  - **Specific**: Year-based rating comparison using the average rating level for each year.
  - **Measurable**: Uses differences in `rating` or `average_rating` across 2024, 2025, and 2026.
  - **Achievable**: The required `rating` and `review-date` fields are available in the prepared review dataset and dashboard visuals.

- **Relevant**: Supports the analytical problem by showing whether customer satisfaction changes over time and how review behavior may align with perceived product quality.
- **Time-bound**: Evaluated across the yearly periods 2024, 2025, and 2026.

1.3.2 Segment Clustering and Revenue Contribution

- **Objective 1: Clustering Quality and Segment Identification** - Use the silhouette score comparison bar chart and the cluster scatter chart to evaluate the clustering solution and identify distinct Nike product groups from February 20, 2026 to March 20, 2026, in order to show that the dataset can be meaningfully segmented into different sales-performance patterns.
  - **Specific**: Compare clustering quality across candidate cluster settings using `silhouette_score`, then visualize product distribution by `segment_id` on the scatter chart.
  - **Measurable**: Uses differences in `silhouette_score` across cluster options and visible separation of products by `segment_id` on the scatter plot.
  - **Achievable**: All required fields are available in `segmentation_metrics.csv` and `product_segments.csv`.
  - **Relevant**: Supports the analytical problem by showing that products can be grouped into meaningful clusters before interpreting sales behavior.
  - **Time-bound**: Evaluated within the analysis window from 20-02-2026 to 20-03-2026.
- **Objective 2: Cluster Revenue Contribution Interpretation** - Use the cluster scatter chart together with the pie chart of revenue contribution by cluster to interpret how each product segment contributes to overall Nike sales performance from February 20, 2026 to March 20, 2026, in order to explain which clusters dominate total revenue and which clusters represent smaller but distinct performance groups.
  - **Specific**: Compare revenue contribution across `segment_id` and relate each cluster's share of revenue to its position in the cluster scatter chart.
  - **Measurable**: Uses % revenue contribution by cluster, cluster membership on the scatter chart, and differences in `avg_daily_revenue` or `avg_total_period_revenue`.
  - **Achievable**: The required fields are available from `product_segments.csv` and `segment_profiles.csv`.
  - **Relevant**: Directly supports the analytical problem by linking clustered product characteristics to overall sales performance contribution.
  - **Time-bound**: Limited to the same analytical period from 20-02-2026 to 20-03-2026.

2 Data Collection, Construction, and Preprocessing

2.1 Product Collection

Note: The extraction procedures described in this section were programmatically implemented and executed via the ***products.py*** script.

2.1.1 Objectives and Approach

- **Objective**: To systematically collect all product metadata from a specific flagship store on an e-commerce platform (e.g., Lazada). The target data fields include product name, selling price, quantity sold, rating score, and inventory status.
- **Approach**: The system utilizes a Hybrid Web Scraping methodology. This approach combines browser automation (via Selenium) to bypass initial security mechanisms (like Captchas) with direct HTTP requests to optimize data extraction speed and efficiency by querying hidden internal APIs.

2.1.2 Detailed Execution Steps

- **Step 1: Session Initialization & Authentication**
  - The system initializes a automated browser instance (Google Chrome) using **Selenium** and **WebDriverManager**.
  - It navigates to the target store's homepage. At this point, the execution thread is intentionally paused to allow for manual human intervention. This is necessary to solve complex Captchas or perform account authentication if required by the platform.
  - Upon successful manual authentication, the system extracts the session **Cookies** and the browser's **User-Agent** string.
  - The automated browser is then terminated to free up system memory, and the process transitions to a lightweight HTTP request mode.
- **Step 2: Category Discovery (Structure Extraction)**
  - Utilizing the extracted Cookies and User-Agent, the system simulates an asynchronous HTTP request (AJAX/XMLHttpRequest) directed at the platform's filter API.
  - The server responds with a JSON payload containing the complete categorical architecture of the store.
  - The system parses this JSON to isolate the list of product categories and their corresponding internal identifier codes (values), which serve as inputs for the next phase.
- **Step 3: Data Extraction**
  - **Automated Pagination**: The system iterates through each discovered category. Within a category, a while loop is employed to navigate through sequential product

pages (page 1, page 2, etc.) by dynamically modifying the page parameter in the API URL.

  - **Data Parsing**: For each queried page, the system receives a JSON array of products. The parser extracts core variables: *Item ID*, *Product Name*, *Selling Price*, *Original Price*, *Rating Score*, *Review Count*, *Warehouse Location*, *Stock Status*, *Brand Name*, and *the Product URL*.
  - **Data Cleaning**: Unstandardized data formats are processed immediately. For instance, text strings representing sales volume (e.g., "1.5k đã bán") are converted into standard integer formats (1500) using regular expressions to facilitate downstream quantitative analysis.
  - **Anti-bot Evasion Mechanism**: The script implements randomized delays (time.sleep between 2 to 3 seconds) between page requests. This simulates organic human browsing behavior, thereby minimizing the risk of IP blocking or rate-limiting by the platform's anti-scraping firewalls
  - The pagination loop terminates when the API returns an empty product list, indicating the end of the category.

- **Step 4: Data Organization and Storage**
  - The system dynamically generates a hierarchical directory structure based on the current execution date (data/MM-DD-YYYY).
  - The data is simultaneously exported into two formats:
    * **CSV File** (products.csv): Intended for end-users to easily read, filter, and visualize the data using spreadsheet software like Microsoft Excel.
    * **JSON File** (products.json): Preserves the raw structured format, acting as the requisite input file for the subsequent phase of the project (Review Collection Process).

2.2 Review Collection

Note: The extraction procedures described in this section were programmatically implemented and executed via the ***reviews.py*** script.

2.2.1 Objectives and Approach

- **Objective**: To systematically extract comprehensive customer feedback, including text reviews, star ratings, and timestamps, for all previously identified products to facilitate product quality analysis.
- Approach: The system employs an API reverse-engineering technique targeting the platform's MTOP architecture. This methodology relies on dynamic cryptographic token generation (MD5 signing), coupled with a local checkpoint system and automated Captcha handling via Selenium, ensuring continuous and resilient data extraction.

2.3 Detailed Execution Steps

- **Step 1: Data Initialization and Checkpointing**

  - The system parses the *products.json* file generated in the first phase. To optimize execution time and prevent data duplication in the event of network interruptions, it implements a checkpoint mechanism. It compares the input list against the existing *reviews.json* output file to bypass products that have already been successfully scraped.

- **Step 2: Session Initialization and Token Extraction**
  - A Selenium-driven browser is temporarily launched to navigate to a sample product page. The primary purpose of this action is not to parse the HTML document, but to force the e-commerce server to issue a specific session cookie (_m_h5_tk). This cookie acts as a fundamental security token for authenticating subsequent API requests.

- **Step 3: Cryptographic Signature Generation (MD5)**
  - To bypass the platform's API security layer, the system dynamically generates an MD5 hash signature for every request. This signature is constructed by concatenating the extracted security token, the current Unix timestamp, a hardcoded AppKey, and the stringified JSON payload containing the request parameters (e.g., **itemId** and **pageNo**).

- **Step 4: Data Extraction and Pagination**
  - The script iterates through the remaining product IDs. For each product, it sends signed HTTP GET requests to the review API, automatically paginating through the results (handling up to a predefined limit, such as 50 pages or 1000 reviews per item). The parser extracts core variables from the server's JSON response, specifically the *Product ID*, *Star Rating*, *Review Content*, and *Timestamp*.

- **Step 5: Real-time Persistence and Anti-bot Evasion:**
  - To mitigate the risk of data loss, the extracted reviews are appended to the local storage immediately after processing each individual product. Furthermore, if the system encounters a rate-limit or an anti-bot block signal (e.g., returning a FAIL_SYS_USER_VALIDATE error), it halts the loop, automatically reopens the Selenium browser to allow for manual Captcha resolution, regenerates the security tokens, and seamlessly resumes the crawling process from the exact point of interruption.

2.4 Data Preparation

This section outlines the data cleaning, and feature engineering procedures. Please note that the subsequent Exploratory Data Analysis (EDA) and its visualizations are documented in a separate supplementary file (***eda.ipynb***)

2.4.1 Product Metadata Dataset

This dataset captures the overarching catalog details and aggregate performance metrics for each item in the flagship store. Each row represents a unique product.

- **id (Primary Key)**: The unique alphanumeric identifier assigned to the product by the e-commerce platform.
- **ten_san_pham (Product Name)**: The full, descriptive title of the item as displayed on the storefront.
- **danh_muc (Category)**: The specific classification of the product (e.g., specific shoe lines, apparel categories) scraped directly from the store's filter menu.
- **gia_ban (Selling Price)**: The current transactional price of the item, reflecting any active discounts.
- **gia_goc (Original Price)**: The baseline retail price of the item before any promotional deductions.
- **so_luong_da_ban (Sales Volume)**: A quantitative integer representing the historical number of units sold.
- **diem_danh_gia (Rating Score)**: The aggregated average star rating for the product, ranging from 1.0 to 5.0.
- **so_luot_nhan_xet (Review Count)**: The total number of customer ratings the product has accumulated.
- **dia_diem_kho (Warehouse Location)**: The geographical origin or fulfillment center location for the item.
- **ton_kho (Inventory Status)**: A categorical indicator specifying whether the item is currently "Còn hàng" (In-Stock) or "Hết hàng" (Out-of-Stock).
- **thuong_hieu (Brand)**: The brand name associated with the item .
- **link_san_pham (Product URL)**: The fully qualified hyperlink to the product's detail page.

2.4.2 Data Cleaning and Standardization For Product Dataset

- **Missing Value Imputation**: Null review counts (so_luot_nhan_xet) were filled with 0, while missing rating scores (diem_danh_gia) were kept as NaN to prevent skewing the mean during statistical analysis.
- **Data Type Casting**: Converted financial and metric columns from text to strict numeric formats (Float/Integer) to enable mathematical calculations.
- **Geographical Standardization**: Merged naming inconsistencies in the warehouse location (dia_diem_kho), specifically mapping "Hồ Chí Minh" to the standardized "Thành phố Hồ Chí Minh".
- **Feature Engineering**:
  - **Objective**: Transform cumulative static data (total items sold) into dynamic daily metrics, synchronizing the results directly back to the source files (JSON and CSV).

- **Baseline Initialization (02-19-2026)**: As the starting point with no prior data, daily sales (so_luong_ban_trong_ngay) for all items are initialized to 0.
- **Calculation Logic**:
  * Calculated by subtracting the historical maximum recorded in the tracker from the newly scraped cumulative total.
  * Derived by multiplying the newly calculated daily sales by the item's current selling price.

2.4.3 Customer Reviews Dataset

This dataset contains granular, user-generated feedback for the scraped products. Each row represents an individual customer review.

- **id_sp (Foreign Key)**: The product identifier, which maps directly back to the *id* in the Product Metadata dataset, establishing a one-to-many relationship.
- **rating (Star Rating)**: The specific ordinal score (typically 1 to 5) awarded by the individual customer.
- **comment (Review Text)**: The unstructured, qualitative textual feedback provided by the buyer detailing their experience or product quality.
- **time (Timestamp)**: The exact temporal record of when the review was submitted, allowing for time-series sentiment analysis.

2.4.4 Data Cleaning and Standardization For Customer Review Dataset

The extracted review texts and numerical ratings were retained in their original state without any additional cleaning or standardization.

3 Data Analysis

References